

Exercício - Mini Serializador de Objetos usando

Objetivo

Neste exercício você irá construir um **mini mecanismo de serialização manual**, utilizando **Reflection em C#**, capaz de transformar qualquer objeto em uma representação textual simples.

A ideia é simular, em pequena escala, o que frameworks como **Entity Framework**, **ASP.NET**, **AutoMapper** e **Serializadores JSON** fazem internamente.

Você deverá inspecionar o objeto em tempo de execução, descobrir suas propriedades dinamicamente e convertê-las para texto obedecendo regras específicas.

Problema

Implemente o seguinte método:

```
string Serializar(object obj)
```

Este método deve retornar uma string no formato:

```
Nome=Pedro;Ano=sétimo
```

Ou seja:

- Cada propriedade deve virar um par `Chave=Valor`
- Os pares devem ser separados por `;`

Requisitos Obrigatórios

Usar Reflection

Você **não pode** acessar as propriedades diretamente (ex: `obj.Nome`).
Você deve:

- Obter o `Type` do objeto
- Usar `GetProperties()` para acessar as propriedades públicas
- Usar `GetValue()` para obter os valores

Serializar apenas propriedades públicas

Somente propriedades públicas devem ser incluídas.

Propriedades privadas ou protegidas não devem aparecer.

Ignorar propriedades com atributo `[Ocultar]`

Crie um atributo customizado:

```
[AttributeUsage(AttributeTargets.Property)]
public class OcultarAttribute : Attribute
{
}
```

Se uma propriedade tiver esse atributo, ela **não deve ser incluída na serialização**.

Exemplo:

```
public class Aluno
{
    public string Nome { get; set; }

    [Ocultar]
    public string Documento { get; set; }
}
```

Saída esperada:

Nome=Pedro

Tratamento especial para ENUM

Se a propriedade for um `enum`, você deve:

- Verificar se o valor possui `[Display(Name = "...")]`
- Se tiver, usar o valor definido no atributo
- Caso contrário, usar `ToString()`

Exemplo:

```
public enum AnoEscolar
{
    [Display(Name = "sétimo")]
    Setimo = 7
}
```

Saída:

Ano=sétimo

O método deve funcionar para qualquer classe

Seu método deve ser genérico o suficiente para funcionar com qualquer tipo:

```
Serializar(new Produto(...));  
Serializar(new Cliente(...));  
Serializar(new Pedido(...));
```

Sem precisar modificar o código do serializador.

Exemplo Completo Esperado

Dada a classe:

```
public class Aluno  
{  
    public string Nome { get; set; }  
    public AnoEscolar Ano { get; set; }  
    public DateTime DataNascimento { get; set; }  
  
    [Ocultar]  
    public string Documento { get; set; }  
}
```

A saída esperada deve ser algo como:

```
Nome=Pedro;Ano=sétimo;DataNascimento=2010-05-20
```

BÔNUS

◆ Bônus 1 — Formatação de DateTime

Se a propriedade for `DateTime`, formate no padrão:

```
yyyy-MM-dd
```

Exemplo:

```
DataNascimento=2010-05-20
```

Bônus 2 — Suporte a objetos aninhados

Se o objeto possuir uma propriedade que também seja um objeto (exemplo: `Aluno.Endereco`), você deve serializar recursivamente.

Exemplo:

```
public class Endereco
{
    public string Rua { get; set; }
    public int Numero { get; set; }
}

public class Aluno
{
    public string Nome { get; set; }
    public Endereco Endereco { get; set; }
}
```

Saída esperada:

```
Nome=Pedro;Endereco.Rua=Rua A;Endereco.Numero=123
```

Restrições Importantes

- Não pode usar bibliotecas externas (Newtonsoft, System.Text.Json, etc.)
- Não pode usar `dynamic`
- Não pode usar `if` baseado em tipo concreto específico (ex: `if (obj is Aluno)`)
- Pode usar `IsEnum`, `GetCustomAttribute`, `PropertyType`, etc.
- Pode criar classes auxiliares (ex: `EnumHelper`)

Conceitos que você deve aplicar

- `GetType()`
- `GetProperties()`
- `PropertyInfo`
- `GetValue()`
- `GetCustomAttribute<T>()`
- `IsEnum`
- Recursividade (bônus)

Entrega Esperada

Seu projeto deve conter:

- Classe `MiniSerializer`
- Classe `OcultarAttribute`
- Helper para enums (ou lógica equivalente)
- Classe(s) de teste (ex: `Aluno`, `Produto`, etc.)
- Um `Program.cs` demonstrando o funcionamento

Dica

1. Cuidado com propriedades `null`.
2. Ignore indexadores.
3. Cuidado com tipos complexos no bônus (evite loops infinitos).
4. Teste com pelo menos dois tipos diferentes.